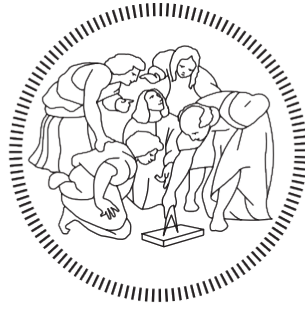




POLITECNICO

MILANO 1863

Academic Year 2025/2026

Data Information Quality Project

Assigned dataset: 11

Stefan Calugaru

Personal Code: 10852010 - Student ID: 212886

Stefano Molteni

Personal Code: 10894104 - Student ID: 306650

Gabriele Pedesini

Personal Code: 10840961 - Student ID: 304425

Table of Contents

1	Introduction	2
1.1	Setup Choices	2
1.2	Libraries	2
2	Pipeline Implementation	4
2.1	Data Profiling	5
2.2	Data Cleaning and Transformation	6
2.2.1	Transformation	6
2.2.2	Association rule	7
2.2.3	Missing values	7
2.2.4	Outliers detection	8
2.2.5	Duplicated detection	8
3	Results and Conclusions	9

1 Introduction

This report presents a structured analysis of the Data Information Quality (DIQ) project, whose main objective is to assess, monitor, and improve the quality of a real world dataset. The motivation behind the project is rooted in the observation that low quality data directly affects the reliability of analytical results and downstream decision making processes, especially when advanced techniques such as machine learning are applied.

For this reason, the project does not focus only on what transformations and cleaning steps were applied, but also on why each design choice was made. Particular attention is given to the justification of methodological decisions, as data quality improvements must be driven by clear assumptions and measurable goals.

1.1 Setup Choices

The project was developed using Visual Studio Code and implemented as a Jupyter Notebook (.ipynb). This choice was deliberate for several reasons. First, the notebook format allows a clear integration of code, results, and explanations, which is essential for reproducibility and transparency. Second, it is consistent with the tools used during course exercises, allowing us to directly apply best practices discussed during lectures.

1.2 Libraries

The implementation relies on a carefully selected set of libraries, each chosen for its specific contribution to data quality assessment:

1. **pandas** is a library for structured data manipulation and cleaning. It is selected because it provides robust support for handling missing values, duplicates, and transformations that are fundamental to data quality tasks.
2. **numpy** is the core library for numerical computing in Python.
3. **matplotlib** is a low-level visualization library for creating customizable plots. It is selected because it allows detailed visual inspection of data distributions and anomalies.
4. **seaborn** is a high-level statistical visualization library built on top of matplotlib. It is selected because it simplifies the identification of patterns, correlations, and outliers relevant to data quality assessment.
5. **ydata-profiling** is a library for automated data profiling and exploratory analysis. It is selected because it generates comprehensive reports.

6. `ipywidgets` is a library for creating interactive controls in Jupyter notebooks.
7. `setuptools` is a library for packaging and distributing Python projects. It is selected because it ensures reproducible installation and deployment of data quality pipelines.
8. `mlxtend` is a library that provides extensions and utilities for machine learning workflows.
9. `scikit-learn` is a comprehensive library for machine learning and data preprocessing.
10. `recordlinkage` is a specialized library for record linkage and entity resolution.

2 Pipeline Implementation

The project follows a standard data quality pipeline, structured to progressively reduce uncertainty and inconsistency in the dataset:

1. **Data Collection:** loading and initial inspection of the raw dataset.
2. **Data Profiling:** identification of attributes, data types, structural constraints, duplication, and completeness. Computation of descriptive statistics to capture distributions, central tendencies, and variability. General info about the dataset, such as number of non-null entries and data types for each column and descriptive statistics about the numeric columns of the dataset.
3. **Data Cleaning and Transformation**
 - **Standardization and normalization:** transformation of data into a consistent format and structure.
 - **Error correction:** identifying and fixing inaccurate or invalid data entries.
 - **Duplicate detection:** finding and resolving duplicate records across the dataset.
4. **Data quality assessment:** evaluating the cleaned dataset to ensure it meets quality standards.

2.1 Data Profiling

The initial data exploration focused on understanding the semantic meaning and data type of each column.

The observation of the dataset allowed the following considerations:

- ZD takes values from 1 to 9 and identifies the different zones of the city of Milan;
- "Superficie somministrazione" is the only quantitative column and represents the surface area, in square meters, of each establishment;
- "Tipo esercizio storico pe" and "Settore storico pe" were observed to be related: the first indicates the general type of establishment, while the second provides a more detailed description.

Based on the available data, data profiling operations were performed, including the following single-column analyses:

1. **Cardinality:** evaluation of the number of distinct values for each column.
2. **Uniqueness:** analysis of the number of rows in the dataset and assessment of column-level uniqueness.
3. **Distinctness:** analysis of how values are distributed across distinct categories.
4. **Constancy:** identification of dominant or constant value in "Superficie somministrazione" column.
5. **Value distribution:** inspection of the extremes values and central tendency measures in the "Superficie somministrazione" column.

At the end of this process, a comprehensive profiling report of the dataset was generated, providing an overall summary of data quality and structural characteristics.

Data profiling confirmed the results of the exploratory analysis. Most columns are categorical and show low distinctness and uniqueness. The columns "Ubicazione" and "Settore storico pe" have the highest percentage of unique values, due to their aggregation of multiple information in semi-formatted form.

2.2 Data Cleaning and Transformation

Cleaning and transformation steps were designed to enforce consistency, reduce redundancy, and improve analytical usability. Each intervention was justified by issues identified during profiling.

2.2.1 Transformation

Several columns were standardized or corrected to resolve inconsistencies:

Column	Transformation applied
<i>Tipo via</i>	Rows where the address type was not consistent with the value reported in <i>Ubicazione</i> were removed (55 rows dropped in this way). Wrong categories in "Tipo via" column were fixed: "VIE" becomes "VIA" and "LARGO" becomes "LGO". This decision prioritizes data coherence over dataset size, as conflicting address information undermines record reliability.
<i>Descrizione via</i>	Rows where the street name was not consistent with the information in "Ubicazione" were removed (52 rows dropped in this way). This decision prioritizes data coherence over dataset size, as conflicting address information undermines record reliability.
<i>Civico</i>	Rows where the street number was not consistent with the value reported in "Ubicazione" were removed (4 rows dropped in this way). This decision prioritizes data coherence over dataset size, as conflicting address information undermines record reliability.
<i>ZD</i>	Rows where the zone identifier was not consistent with <i>Ubicazione</i> were removed (29 rows dropped in this way). This decision prioritizes data coherence over dataset size, as conflicting address information undermines record reliability.
<i>Ubicazione</i>	After checking in which ways street number is defined in "Ubicazione". Then, street number formatting was standardized by replacing all occurrences of "num." with "N." (162 rows updated). This decision enable reliable parsing and matching.
<i>Tipo esercizio storico pe</i>	Redundant and overly specific categories were replaced with a reduced set of standardized values. This decision improve interpretability and reduce sparsity.
<i>Settore storico pe</i>	All unique values in the Settore storico pe column were first enumerated together with their frequencies, then converted to lower-case and further decomposed by splitting each entry on commas and semicolons. Similar categories were merged through value substitution. New binary columns were generated for each category, after which the original column was removed. This transformation enabled its use as input for machine learning models.
<i>Insegna</i>	The column was removed due to a high percentage of missing values and limited analytical relevance.

2.2.2 Association rule

Association rule mining was applied using the fpgrowth algorithm. The motivation behind this choice is twofold: fpgrowth is computationally efficient, and association rules provide interpretable relationships that can be directly used for data imputation.

The extracted rules revealed strong and consistent dependencies between commerce related attributes, such as:

- (Forma commercio = solo somministrazione) $\rightarrow$ (Forma commercio prev = somministrazione)
- (Forma commercio prev = somministrazione) $\rightarrow$ (Forma commercio = solo somministrazione)
- (Tipo esercizio storico pe = bar) $\rightarrow$ (Forma commercio prev = somministrazione)
- (Forma vendita = misto) $\rightarrow$ (Forma commercio prev = somministrazione)

These rules were later exploited to justify rule based filling of missing values instead of relying exclusively on statistical or model based imputation.

2.2.3 Missing values

Detection and quantification of missing or null values across attributes. Different strategies were adopted depending on the nature of the attribute:

- Missing values in "Civico" column were filled using the number found after "N. " in the "Ubicazione" column. Then, the "Ubicazione" column was dropped, since all its information was redundant with other columns.
- The mean of the "Superficie somministrazione" column was computed and used to fill its missing values with the integer mean value. This choice preserves the global distribution while avoiding the introduction of extreme values.
- Missing values in the "Forma commercio" column were imputed starting from the association rules found before.
- Linear models were used to impute missing values in the "Tipo esercizio storico pe", "Forma vendita", "Forma commercio", "Forma commercio prev" columns. In particular, were used the boolean columns extracted from "Settore storico pe" as parameters. This approach was chosen to capture multivariate relationships that simple univariate methods would ignore.

2.2.4 Outliers detection

The analysis of the value distribution of "Superficie somministrazione" showed that values range from 2 to 2336 square meters, with most observations below 250. Descriptive statistics and the histogram highlight the presence of a small number of large values; however, all observed values were considered reasonable, and no true outliers were identified.

2.2.5 Duplicated detection

Duplicate detection was carried out step by step to ensure accuracy:

1. Exact duplicate records were first identified and removed.
2. A new *Ubicazione* attribute was created by combining address-related fields to obtain a consistent address representation.
3. Record linkage methods were then used to find possible duplicates with small differences, using similarity measures.
4. Only record pairs with many matching attributes and the same street number were kept, in order to reduce false matches.
5. The identified duplicates were removed, and the temporary *Ubicazione* column was dropped after use.

This approach ensured reliable duplicate removal while limiting the risk of incorrectly matching different records.

3 Results and Conclusions

To measure the effect of data quality improvements, a machine learning model was trained on both the original (dirty) dataset and the cleaned dataset. This approach provides a practical way to evaluate data quality, since better data should lead to better model performance.

A supervised learning approach was used so that clear evaluation metrics such as precision, recall, and F1 score could be applied. The `KNeighborsClassifier` from the `scikit-learn` library was chosen because it is simple and easy to interpret.

The results show a clear improvement after data cleaning:

- Precision increased from 4% to 46%
- Recall increased from 20% to 63%
- F1 score increased from 6% to 51%

The very poor performance on the dirty dataset was mainly caused by missing values, inconsistent category labels, and duplicated records. These issues prevented the model from learning useful patterns. After cleaning the data, the model was able to use the features more effectively, which confirms that the data quality improvements were successful.